

SIGReg outside image SSL: calibrating LeJEPA for a world model

An implementation study of LeJEPA applied to action-conditioned video, with the corrections the objective needs in that setting

AQ-Mario · Super Mario Bros 1-1 · 8 September 2026 · every number measured, none estimated

Summary. We implement LeJEPA's SIGReg objective in a domain it was not developed for — an action-conditioned world model over video, evaluated by whether a control-relevant state variable survives in the representation. The objective transfers, but not unmodified. We report (i) a **closed-form null** for the SIGReg statistic, without which its value and therefore λ are not comparable across batch sizes; (ii) its collapse-detection power is approximately **linear in batch size**, placing it in direct tension with the memory cost of video; (iii) a trailing BatchNorm, which the architecture requires, **reduces its collapse signal 25×**; (iv) temporally correlated batches **break its iid assumption**, making a healthy latent read as collapsed; (v) in this setting the **stop-gradient was still required**, contrary to the simplification claim; and (vi) at the application level, **preventing collapse did not preserve usable state** — an uncollapsed representation scored $R^2 = -0.383$ on the one variable a planner needs.

1. Setup

A 9.8M-parameter JEPA over Super Mario Bros 1-1. ViT-Tiny patch-14 at 224px encodes each frame to a 192-dim latent through an MLP head ending in `BatchNorm(affine=False)` ; a 6-block causal predictor conditioned on actions through AdaLN-Zero predicts the next 5 latents from 3 of context. Trained bf16 on a single H100.

COMPONENT	PARAMS	NOTE
encoder (ViT-Tiny p14 @224)	5.65M	257 tokens/frame
predictor (6 × AdaLN-Zero)	4.04M	causal over the window
action encoder	0.04M	2 frames × 6 buttons → 192
auxiliary heads	0.07M	ablated in the control

Data. 8,000 episodes, 4.88M observations, 27.4 GB compressed. Two behaviour policies: a jump-biased random policy (reaches ~45% of the level) and a PPO explorer trained to the flagpole, because a representation graded only on the opening of a level is graded on the easy half. Ground truth for x, y, scroll and death comes from console RAM, validated against the emulator's own reported values.

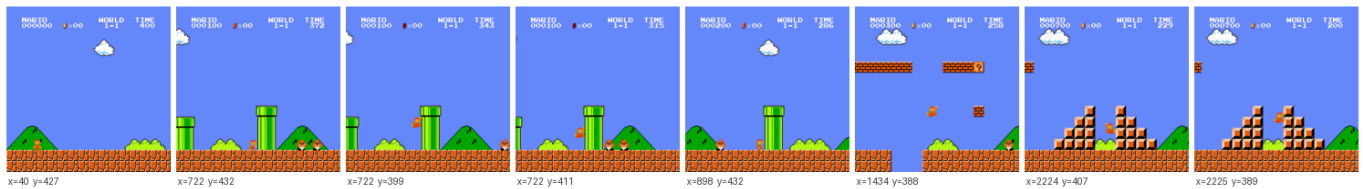


Figure 1. One training episode. `x` is distance along the level and `y` is height on screen, both read from RAM. These are never inputs — they are the probe targets used to evaluate the frozen representation afterwards.

EVALUATION PROTOCOL

Encoder frozen, MLP probes fit on held-out complete trajectories and scored on a disjoint set of 60. Splitting by observation rather than trajectory leaks badly here: consecutive frames are near duplicates. Reported R^2 is against held-out variance and is allowed to go negative.

2. The calibration SIGReg needs and does not ship with

The Epps–Pulley statistic compares the empirical characteristic function of each 1-D projection to the Gaussian one. Under the null, $E|\varphi_n - \varphi|^2 = (1 - e^{-t^2})/n$ exactly, so integrating against the Gauss–Hermite weight gives a closed form for the value a *perfectly* Gaussian embedding produces:

$$E[T \mid z \sim N(0, I)] = \sqrt{\pi} \cdot (1 - 1/\sqrt{2}) / n = \mathbf{0.51914 / n}$$

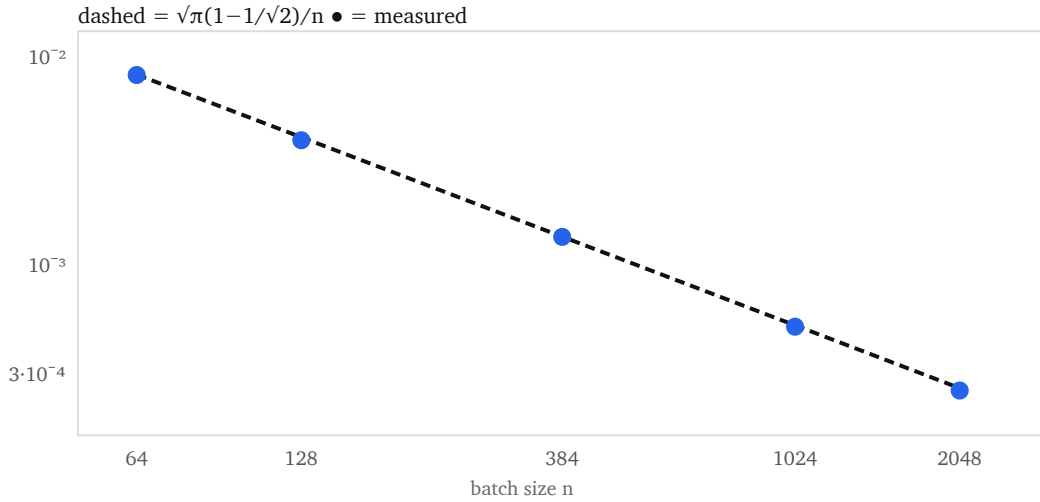


Figure 2. Predicted vs measured null, $n = 64 \dots 2048$. Agreement within 3% throughout.

Three consequences that matter in practice:

- At batch 64 the term **floors at 0.0013** no matter how Gaussian the encoder becomes. Driving SIGReg toward zero is not a goal; reaching $\approx 1.0 \times$ null is.
- Raw SIGReg values from runs at different batch sizes are **not comparable**, so neither are their λ . The comparable quantity is `ratio = T / null(n)`.
- Because the floor is sampling noise, the optimiser can push slightly *below* $1.0 \times$ by making the batch mildly repulsive. That is benign, and a second reason the target is a ratio near one rather than a loss near zero.

3. Its power to see a collapse scales with batch size

	n=32	n=64	n=128	n=512	n=2048
rank 2	2.8×	5.6×	10.9×	41.8×	173.8×
rank 4	1.8×	3.3×	6.4×	25.0×	97.8×
rank 16	0.8×	1.2×	2.0×	7.2×	28.5×
192 (healthy)	0.4×	0.5×	0.5×	1.0×	2.8×

SIGReg / its own null. A detector needs the collapsed rows far above the healthy row.

Figure 3. SIGReg in units of its own null, for a rank- r latent after BatchNorm, $D = 192$.

At $n = 32$ a fully collapsed latent reads $2.8\times$ against a healthy $0.4\times$ — barely separable, so the term cannot resist what it can hardly see. At $n = 512$ the same collapse reads $41.8\times$ against $1.0\times$. This is the mechanism behind §5.

THE TENSION SPECIFIC TO VIDEO

A world-model step encodes `batch × window` images — at window 8 and batch 96 that is 768 images and 197,376 ViT tokens. Memory therefore pushes the batch *down*, into exactly the regime where SIGReg is weakest. Activation checkpointing is not a throughput optimisation here; it is what buys the regulariser its statistical power.

4. A trailing BatchNorm halves the job

The projection head must end in BatchNorm rather than LayerNorm: a trailing LayerNorm places every embedding on a fixed-radius sphere, which fights the isotropic-Gaussian target directly. But BatchNorm pins the covariance *diagonal* for free, and that is a large part of what SIGReg was detecting.

RANK-8 LATENT	SIGREG / NULL
raw	254×
after BatchNorm(affine=False)	10×

Still unambiguous, but $25\times$ less signal. BatchNorm and SIGReg are complementary rather than reinforcing — one fixes the diagonal, the other the rest — and SIGReg values reported from a BN-projected encoder are a different quantity from those without one.

The corresponding positive result, and the single experiment that says the design is sound: SIGReg escapes a rank-8 collapse **only when the gradient flows through the BatchNorm**, as it does in a real encoder. Optimising the latent as a free tensor barely moves it.

STEP	SIGREG	× NULL	EFF. DIM
0	0.0139	10.0	7.9
100	0.0051	3.8	21.5
300	0.0021	1.6	52.0
600	0.0015	1.1	69.5

5. In this domain the stop-gradient was still required

The headline simplification — that SIGReg removes the need for stop-gradients and EMA teachers — did not hold at our scale. Identical settings, 500 steps, varying only the stop-gradient on the target latents:

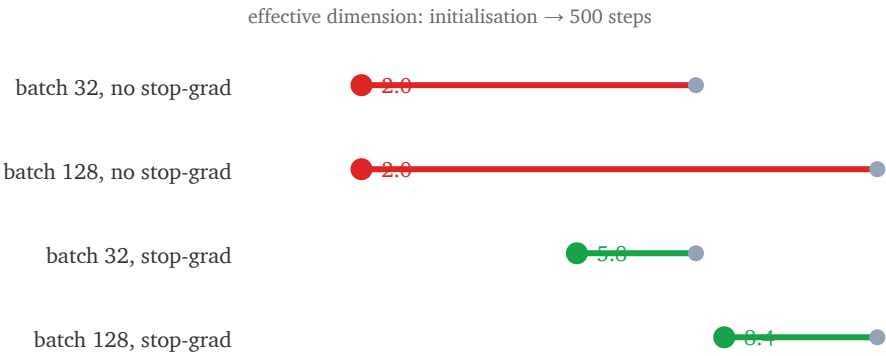


Figure 4. Effective dimension, initialisation → 500 steps. Without the stop-gradient both batch sizes collapse to rank 2 while reporting a 5-step gain of 0.97.

λ alone could not substitute. Sweeping it without the stop-gradient gave effective dimensions of 1.08, 1.89 and 3.50 at $\lambda = 0.1, 1$ and 5 — all collapsed. The arithmetic says why:

λ	PREDICTION LOSS SAVED BY COLLAPSING	SIGREG COST	AUX COST	RATIO
0.1	0.796	0.010	0.008	45×
1.0	0.778	0.065	0.008	11×
5.0	0.745	0.181	0.008	4×

WHY A WORLD MODEL IS A HARDER CASE THAN IMAGE SSL

With a shared encoder, $\text{MSE}(\hat{z}, z_{\text{target}})$ is *directly* minimised by making the target constant, because the target is produced by the same trainable network. Collapse is not a failure mode to be avoided — it is the objective's global optimum unless something removes the incentive. Positive-pair image objectives share this structure, but the prediction task here is far easier to trivialise: one scalar suffices, and a rank-1 latent still has unit variance per coordinate after BatchNorm, so the predictor need only predict that scalar.

With the stop-gradient in place λ becomes an effective lever, and at $\lambda=10$ the effective dimension *doubles* over training rather than falling ($10.3 \rightarrow 21.1$ at batch 96).

6. Temporally correlated batches break the null

A world-model batch is **B windows × W near-identical frames** — two emulator frames apart at skip-2. SIGReg's null assumes iid samples, so the effective count is B, not B×W. Measured on a synthetic *full-rank* latent batched as 32×6:

HOW IT IS SCORED	READS	VERDICT
flattened, against the n=192 null	6.7×	"collapsing"
flattened, against the honest n=32 null	1.2×	fine

Flattening also makes SIGReg penalise the within-window similarity the predictor depends on — it pushes consecutive frames apart while the prediction loss pulls them together. Computing the statistic per frame position and averaging keeps every latent, makes the null exact at $n = B$, and removes the conflict.

The same correction applies to the participation-ratio diagnostic. Marchenko–Pastur gives $E[PR] = nD/(n+D)$, so a genuinely full-rank 192-dim latent reads **128, not 192**, at $n = 384$. Inverting it ($D_{\text{eff}} = PR \cdot n / (n - PR)$) recovers 192.0 ± 1 at every n tested from 128 to 8192, and stays at 7.6 on a rank-8 latent.

A RELATED TRAP, FOUND THE EXPENSIVE WAY

Every downstream measurement runs the encoder in `eval()`, where BatchNorm uses running statistics. Those track pre-BN activations, which drift quickly while the encoder is learning. On a checkpoint taken 10% into a run, the *same* encoder on the *same* data scored a y-probe of **−0.028** with stale statistics and **+0.121** after re-estimating them, with running variance sitting at 0.0975 instead of ≈ 1 . End-of-run measurements are unaffected because the learning rate has annealed and the statistics have caught up — which is precisely why this survives undetected until someone takes a mid-training checkpoint.

7. Preventing collapse did not preserve usable state

This is the application-level result, and an anti-collapse guarantee does not address it. Our SIGReg-only variant **did not collapse** and still could not report the height of the character — the single variable a platformer planner needs most.

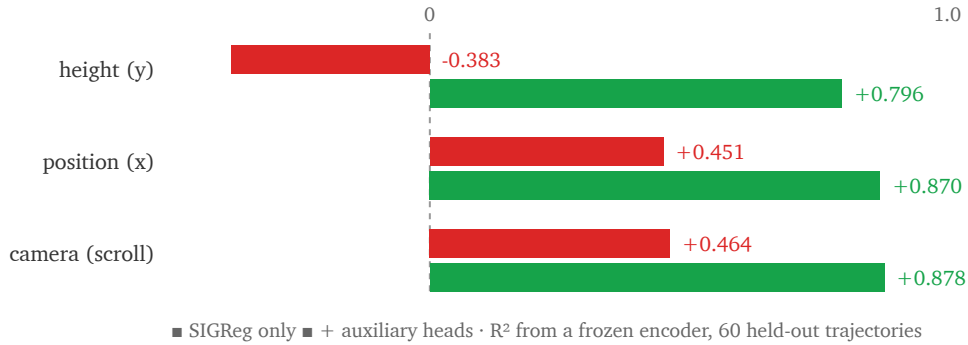


Figure 5. Frozen-encoder probe R^2 , 60 held-out trajectories, full dataset. Both models trained identically apart from the auxiliary loss terms.

PROBE	SIGREG ONLY	+ AUX HEADS (0.07M PARAMS)
height (y)	-0.383	0.796
position (x)	0.451	0.870
camera (scroll)	0.464	0.878
death within 5 steps (AUC)	0.628	0.927
latent distance usable to	0 px	350 px

An isotropic-Gaussian embedding is a **floor, not a sufficient condition**: it guarantees the representation is non-degenerate, not that it contains what the downstream task requires. A 0.07M-parameter auxiliary head — 0.7% of the model — recovers it.

Independent replication of the positive result

The aux model's height probe was re-fit and re-scored on shards collected in a separate run and never seen by training: $R^2 = 0.932$. Controls on the same split:

PREDICTOR OF HEIGHT	R^2
trained encoder (192 dims)	0.932
trained encoder, <i>linear</i> probe only	0.868
raw pixels, 112×112 (12,544 dims)	0.028
random projection of pixels → 192	-0.180
untrained encoder	0.013
trained encoder, shuffled labels (code sanity check)	-0.439

No pixel baseline clears 0.03 at any resolution, so the task is not trivially solvable; the linear probe at 0.868 shows the information is nearly linearly present rather than manufactured by the probe; and held-out frames sit 1.29×

further from training frames than training frames sit from each other, so this is not duplicate leakage.

8. Limitations

- **Training budget.** Every number above comes from 4,000 steps — roughly 4% of the planned schedule, on half the dataset. The SIGReg-only result in §7 is therefore also a statement about a young encoder, and the honest reading of -0.383 is "did not acquire height at this budget", not "cannot". A full-budget control is the obvious next experiment and is not yet run.
- **Single environment, single level.** One game, one level, one visual style. The calibration results in §2–§4 are properties of the statistic and should transfer; §5–§7 are empirical and may not.
- **The aux comparison is not a fair test of representation quality.** The auxiliary variant is *trained* to make height decodable and is then measured for whether height is decodable. It shows that a cheap supervised head fixes the problem; it does not show that the self-supervised objective learns a better representation.
- **The paper's claims are paraphrased here, not quoted.** The measurements are ours; the statements of LeJEPA's position in §5 should be checked against the paper text before any of this is published.
- **In-distribution vs. full-level.** The height probe reads 0.943 when scored on the same behaviour distribution the model trained on and 0.796 across the whole level. The latter is the number reported throughout; the gap is the cost of never having seen the back half of the level.

9. What this suggests for practice

1. Report `SIGReg / null(n)`, never raw SIGReg, and publish the batch size next to λ . The raw value floors at $0.51914/n$.
2. In video or RL, compute the statistic **per frame position**, not over the flattened batch. Otherwise a healthy latent reads as collapsing and the objective fights the prediction task.
3. Treat batch size as a SIGReg hyperparameter, not only a throughput knob. Its collapse-detection power is roughly linear in n .
4. Do not assume the stop-gradient is removable outside image SSL at practical batch sizes. It is one ablation and it was decisive here.
5. Re-estimate BatchNorm statistics before any mid-training evaluation.
6. **Anti-collapse is not representation quality.** Gate on a probe of the state the downstream task actually needs — it costs minutes and it is the only measurement here that a collapsed-but-confident model could not fake.